Promo Finder: Ranked Promo Clips from Microdrama Screenplays

Solution Document | Problem 1: Promo Finder | Repository: github.com/chandra8688/kuku-promo-finder

1. Problem Selected & Why

We picked **Problem 1, Promo Finder**. Vertical microdramas live on short promo clips (20 to 60 s) for Reels, Shorts and TikTok, and finding them means an editor scrubbing 50 to 100 episodes per show. Working from the screenplay means a ranked shortlist exists before the footage is cut, so the editor starts from candidates instead of a blank timeline.

2. Core Assumptions

- **Hook in 3 seconds.** The opening 3 s must carry a conflict, a provocative line, a loud SFX or a physical action.
- **Ends unresolved.** The final beat leaves an open question; a clip whose tension resolves is not a promo.
- **One continuous span.** No stitching across episodes. A span may cross a scene boundary inside an episode; such crossings are flagged for the reviewer, not blocked.
- **Screenplay, not footage.** The system scores screenplay promo potential. Acting, shot quality, music and edit rhythm are invisible to it, and nothing here measures click-through or retention.
- **The deliverable is show-level.** The question is which promos to post first across the whole show, so ranking is global with at most 2 clips per episode.

3. How the Solution Works

- **S1 Parse** (`src/parse.py`, `src/speaker.py`): turns `.txt` screenplays into beats (DIALOGUE, ACTION, SFX, CAMERA) with centisecond timings; handles speaker headers with several parentheticals, which previously dropped the emotion label on 56 beats across 24 episodes.
- **S2 Annotate** (`src/annotate.py`): one model call per episode, whole episode in context, scoring each beat for hook (h), open question (q), resolution (r), context need (n) and intensity (t). The model returns beat IDs, never timestamps; code owns all timings. Beats the model fails to score are counted as missing, not as zero, and make any clip containing them low-confidence.
- **S3-S4 Candidate engine** (`src/engine.py`): enumerates every 20 to 60 s run of consecutive beats and scores the whole span: strongest hook in the clip's own first 3 s (including SFX, action and camera cues), strongest open question in the last 6 s, resolution, context need, duration-weighted intensity and position. Hard gates are structural only; narrative weakness lowers the score instead of deleting the clip. `run.py`, the evaluation and the weight fit all call this one module with one config, `config/scoring_v2.yaml`, whose hash is written into every run.
- **S5 Judge** (`src/judge.py`): its scores do not affect ranking (weight 0.0, chosen on evidence: folding them in cut test top-1 from 5/5 to 2/5). It removes a clip only when it and S2 both say the tension resolves, logged in `judge_rejected.json`, and it writes the one-line "why".
- **S6 Rank** (`run.py`): top 15 by S3 score, at most 2 per episode, low-confidence clips last, ties broken down to candidate ID.
- **S7 Refine** (`src/refine.py`): after the ranking is frozen, trims dead lead-in and ends the cut just after the last dialogue, SFX or action beat. Every adjustment is re-checked against the duration limits and the rule that no cut may slice through a line.

4. How It Scales to Every Show

- **Linear call count.** One annotation call per episode plus batched judge calls; the pairwise alternative was estimated at 22,350 calls for 50 episodes. Measured on the 50-episode run: 100 model calls, 379,640 input tokens, 410.0 s wall clock (`run_manifest.json`).
- **Runs without a key.** `--no-llm` runs end to end with 0 network calls. That mode is the Tier-0 heuristic (baseline B3 below), not the full pipeline.
- **Config, not code.** Weights and thresholds live in YAML and every candidate file records which config scored it. Not yet tested on genres other than this show.
- **Reproducible where it can be.** Parsing, search, gates, refinement and ranking give identical output for identical input and config. LLM annotation does not; see section 6.

5. How We Know It's Good

Reserved test set: 5 episodes, 7 reference windows. These windows were proposed and reviewed by a model and never verified by a person, so read the numbers as agreement with an AI-assisted reference. The set has been evaluated four times (default weights, unconstrained dev fit, bounded dev fit, and the bounded fit after this revision's engine fixes); all four are in `eval_report.md` and the last is below.

System	Top-1	Recall	Med start err	Med end err
V2 (ours)	4/5	6/7	5.55 s	4.98 s
B1 last 45 s	5/5	5/7	7.50 s	5.45 s
B3 Tier-0 heuristic	2/5	5/7	5.20 s	6.10 s
B2 random, 20 seeds	1.3/5	4.9/7	n/a	n/a

Dev set: 12 episodes, 25 human-labelled windows. The weights were fitted on this set, so these numbers are in-sample and favour V2 over the untuned baselines.

	V2 (in-sample)	B1 last 45 s	B3 Tier-0
Top-1	11/12	12/12	6/12
Recall	24/25	12/25	21/25

Show-level precision@15: 13 of 15 approved, labelled by Chandra Shekar in one pass, reading each clip's screenplay beats with the model's scores hidden (`scripts/review.py`). B1 has no score, so it cannot produce a show-level ranking at all.

6. Where It Fails

- **B1 ties us on per-episode top-1**, because writers put the cliffhanger at the end of every episode. Our case rests on mid-episode promos B1 never returns and on the show-level ranking it cannot make.
- **The evidence is thin.** Five test episodes cannot separate systems (see the confidence intervals in `eval_report.md`), and the test references are AI-reviewed.

- **Annotation jitter.** Across three independent runs, exact beat agreement was 72 to 78% on hook and 75 to 83% on open question, and the top-1 clip agreed in 6 of 8 comparable episodes. The shipped runner annotates once; a k-sample aggregation helper exists (`src/consistency.py`) but is not wired in.
- **Small-sample fitting.** An unconstrained fit on 25 windows flipped penalty signs and rewarded resolved endings. The fit is now bounded: penalties stay at or below 0, hook and cliffhanger at or above 1.0.
- **Our own pipeline drifted.** This revision found `run.py` scoring with a different config from the one evaluated; on synthetic episodes the two agreed on top-1 in 1 of 12. That is fixed, and a regression test pins production, evaluation and tuning to one engine and one config.

Integrity: the test suite (`python -m pytest tests/ -q`), the leak check (`python scripts/leak_check.py`) and the submission check (`python scripts/check_submission.py`) all pass.